CO5_ASSESSMENTS 2

PSEUDOCODE WRITING TASK

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

1. Real-Time Face Detection and Recognition

Pseudocode

```
ALGORITHM: REAL_TIME_FACE_DETECTION_AND_RECOGNITION

INPUT:
   Live camera/video stream
   Trained face recognition dataset

OUTPUT:
   Video frames with face bounding boxes and identity labels

BEGIN

1. INITIALIZATION

   Load OpenCV library

   Initialize camera
      CAMERA ← OpenCamera()

   IF CAMERA is not available THEN
      Display "Camera not found"
      STOP
   END IF

   Load trained face detection model
      FACE_DETECTOR ← LoadFaceDetector()

   Load trained face recognition model
      FACE_RECOGNIZER ← LoadRecognitionModel()

   Load registered persons and their identity labels
      REGISTERED_DATA ← LoadTrainedDataset()

   Set recognition threshold
      THRESHOLD ← predefined value

   Set frame processing interval
      FRAME_SKIP ← 2

   FRAME_COUNT ← 0
```

2. START CONTINUOUS VIDEO PROCESSING

WHILE CAMERA is open DO

FRAME ← CaptureFrame(CAMERA)

IF FRAME is empty THEN
BREAK
END IF

FRAME_COUNT ← FRAME_COUNT + 1

3. EFFICIENT FRAME CONTROL

IF FRAME_COUNT MOD FRAME_SKIP ≠ 0 THEN

Display FRAME

IF UserPressed('q') THEN
BREAK
END IF

CONTINUE

END IF

4. PREPROCESSING

Resize FRAME to suitable resolution

Convert FRAME to grayscale
GRAY_FRAME ← ConvertToGray(FRAME)

Apply noise reduction
GRAY_FRAME ← GaussianBlur(GRAY_FRAME)

Important points to mention

Image Acquisition:

The live camera is opened and frames are continuously captured.

Preprocessing:

Frames are resized, converted to grayscale and filtered to reduce noise and improve detection.

Face Detection:

A Haar Cascade or OpenCV DNN face detector detects multiple faces in each frame**.**

Feature Extraction:

Each detected face is converted into suitable facial features using LBPH or a face-embedding model.

Recognition:

Extracted features are compared with the registered dataset. If the distance is below the threshold, the person's identity is displayed; otherwise, the person is classified as **Unknown**.

Visualization:

Bounding boxes and identity labels are drawn using OpenCV functions.

2. Vehicle Detection, Tracking and Counting

Pseudocode

```
ALGORITHM: VEHICLE_DETECTION_TRACKING_AND_COUNTING

INPUT:
   Live traffic video stream

OUTPUT:
   Detected vehicles with bounding boxes,
   tracking IDs,
   vehicle count,
   virtual line crossing information

BEGIN

1. INITIALIZATION

   Load OpenCV library

   Initialize video camera
      CAMERA ← OpenCamera()

   IF CAMERA is not available THEN
      Display "Video source not found"
      STOP
   END IF

   Initialize vehicle detector
      VEHICLE_DETECTOR ← LoadVehicleDetector()

   Initialize vehicle tracker
      TRACKER ← InitializeTracker()
   Define Region of Interest
      ROI ← DefineRoadRegion()

   Define virtual counting line
      COUNT_LINE ← DefineVirtualLine()

   VEHICLE_COUNT ← 0

   NEXT_ID ← 1

   Create empty list of active tracks
```

TRACKS ← EMPTY

Create empty set of counted vehicle IDs
COUNTED_IDS ← EMPTY

2. START VIDEO LOOP

WHILE CAMERA is open DO

FRAME ← CaptureFrame(CAMERA)

IF FRAME is empty THEN
BREAK
END IF

3. PREPROCESSING

Resize FRAME

Apply Gaussian blur

Convert frame if required

Extract Region of Interest

ROI_FRAME ← FRAME[ROI]

4. VEHICLE DETECTION

Important points to mention

Tracking:

Every vehicle receives a unique tracking ID. The system compares current detections with previous tracks using centroid distance, bounding-box overlap or a tracker such as CSRT/KCF.

Counting:

A virtual line is placed across the road. When a tracked vehicle moves from one side of the line to the other, the count is increased.

For example:
Vehicle ID = 5

Previous position → Above line
Current position → Below line

Therefore:

Vehicle 5 crossed line
Vehicle Count = Vehicle Count + 1

Avoiding Double Counting:

The COUNTED_IDS set stores IDs of vehicles that have already crossed the line. If the same vehicle remains in the video for many frames, its ID is already present, so its count is not increased again.

Multiple Vehicles:
Each vehicle has a separate ID, bounding box and centroid. Therefore, several vehicles can be tracked simultaneously.

Real-Time Optimization:
The system can improve speed by:

- Resizing frames.
- Processing only the ROI.
- Using a lightweight detector.
- Tracking instead of detecting unnecessarily.
- Removing lost tracks.
- Avoiding repeated processing of already counted vehicles.

Loop Termination:
The loop terminates when the video ends, the camera becomes unavailable, or the user presses **q**. Finally, the camera is released and OpenCV windows are closed.